

遠隔バイブレーション手順書

Android スマートフォンとスマートバンドを活用した遠隔バイブレーションの手順書です。
サンプルコードはありますが、基本的なことしか書いていないので各々で調べて、工夫して
様々な機能を追加してください。

前提

スマートバンドの接続および Gadget Bridge の設定は済んでいるものとします。

Gadget Bridge の許可設定は Bluetooth コネクト、Bluetooth スキャン、Fine location、

Manage Do Not Disturb、Post Notifications、通知を on あたりが通っていれば問題ないと思
います。

確認事項

1. Gadget Bridge 内の設定→開発者用設定→Intent API → 一番下のデバッグコマンドを
許可
が許可されていることを確認してください。
2. 本体設定→アプリ→Gadget Bridge → アプリのアクセス許可 → その他の権限→
Run commands in Termux environment
が許可されていることを確認してください。

手順

1. [Ternux](#) および [Termux API](#) のインストール

F-Droid から APK ファイルをダウンロードし、それぞれインストール。

F-Droid のアプリはインストールしなくてもよい。

Termux および Termux API はスマホの設定→アプリから通知の許可、バッテリーセー
バーを制限なしに設定。Gadget Bridge も設定してない場合は同様に設定。

※ Termux は Google Play ストアでも表示されるが、そちらはバージョンが古く、使え

ない。



Termux

パッケージを伴ったターミナル・エミュレーター

Termux combines powerful terminal emulation with an extensive Linux package collection.

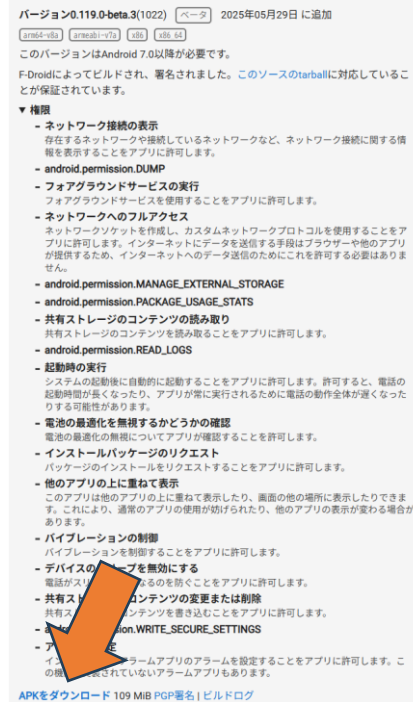
- * Enjoy the bash and zsh shells.
- * Edit files with nano and vim.
- * Access servers over ssh.
- * Compile code with gcc and clang.
- * Use the python console as a pocket calculator.
- * Check out projects with git and subversion.
- * Run text-based games with frotz.

At first start a small base system is downloaded - desired packages can then be installed using the apt package manager known from the Debian and Ubuntu Linux distributions. Access the built-in help by long-pressing anywhere on the terminal and selecting the Help menu option to learn more.

Read help online: <https://wiki.termux.com/>

Reddit Community: <https://termux.com/community>

作者: Tarek Sander - ライセンス: GNU General Public License v3.0 only - ウェブサイト - 問題を報告 - ソースコード - 変更履歴 - ビルドのメタデータ - 再現性の状態



2. 必要なツールのインストール

Termux を起動し、vim や nano 等のエディタをインストールする。

Python もインストールする。

2.x. ssh の設定

このままでも操作は可能だが、Python コードをスマホで書くのは非常に面倒なので、ssh をできるようにすることを推奨。

ssh 設定手順（1～4 は Termux で行う）

1. IP アドレスの確認

以下のコマンドを実行して、スマートフォンの wlan0 の IP アドレスを調べる。

ssh 接続する際に使う。

```
ifconfig
```

2. 必要なパッケージのインストール

以下のコマンドをそれぞれ実行して openssh をインストールする。

```
pkg update
pkg install openssh
```

3. パスワードの設定

以下のコマンドを実行して、パスワードを設定する。

ssh 接続の際に聞かれるので覚えておく。

```
passwd
```

4. ssh サーバーを起動する

以下のコマンドで ssh サーバーを起動する。

これは、スマホの起動毎に毎回必要なので注意。

```
sshd
```

5. PC または Raspberry pi で ssh 接続する。

Termux のポート番号は 22 ではなく 8022 なのでこのように指定して実行する。

```
ssh 調べた IP アドレス -p 8022
```

※この際、VScode のリモートエクスプローラーは使えないのでターミナルや、Ubuntu などを使う。

3. Gadget Bridge 操作用サーバーの設計

今回は、スマートフォンを http サーバーとして立て、request を送る形で、Raspberry pi や PC からスマートバンドへ疑似通知や疑似通話を送信する設計としています。

http サーバーを立てるコードを nano 等のエディタ作成し、保存する。

- 自力で作成する場合は、Gadget Bridge で [Intent](#) として提供されているコマンドや API を確認し、broadcast 等の設定を行うことで疑似通知と疑似通話ができる。
- セキュリティを考えた設計にすること。

サンプルコードでは、基本的な機能として疑似通知、疑似通話、短時間通話によるバイブレーションを使えるようにしています。また、通話の発信者も変数として変更可能で、サーバーの一定時間無操作による自動終了も実装しています。各々工夫して活用してください。

サンプルコードは下にも置きますが、github または [こちら](#)の方がコピペしやすいと思います。

4. Raspberry pi 側で request を送る

Termux で作成したファイルを python で実行し、サーバーを待機状態にさせる。
Raspberry pi で python コードを作成し、実行することでバイブレーション等を実行させる。

サンプルコードでは、request に get で URL を渡し、スマホ側で指定した機能を実行しています。いくらでも工夫の余地があるので、頑張って工夫してみてください。

サンプルコード（スマホ側）

```
import subprocess
import threading
import time

from http.server import BaseHTTPRequestHandler, HTTPServer
import urllib.parse

# -----
# 設定:最後のアクセスから何秒放置されたら終了するか（例: 300 秒 = 5 分）
IDLE_TIMEOUT_SEC = 60
# 最終アクセスの時刻を保持する変数(初期値は現在時刻)
last_access_time = time.time()
server_instance = None

# -----
class RequestHandler(BaseHTTPRequestHandler):
    def address_string(self):
        return self.client_address[0]

    def do_GET(self):
        global last_access_time
        # リクエストが来たら最終アクセス時刻を現在に更新(タイマーリセット)
        last_access_time = time.time()

        parsed_path = urllib.parse.urlparse(self.path)
        path = parsed_path.path
        query = urllib.parse.parse_qs(parsed_path.query)

        try:
            if path == "/start":
                caller_name = query.get("caller", ["Raspberry Pi"])[0]
```

```

        self.vibrate_start(caller_name)

        self.send_ok(f"Vibration STARTED (Caller: {caller_name})")
    elif path == "/stop":
        self.vibrate_stop()

        self.send_ok("Vibration STOPPED")
    elif path == "/vibrate":
        sec = float(query.get("sec", ["1"][0]))
        caller_name = query.get("caller", ["Raspberry Pi"])[0]
        self.vibrate_start(caller_name)

        time.sleep(sec)

        self.vibrate_stop()

        self.send_ok(f"Vibrated for {sec} seconds")
    elif path == "/notify":
        sender = query.get("sender", ["Wall Entity"])[0]
        body = query.get(
            "body", ["I am living in your walls. You may be concerned..."]
        )[0]

        self.send_notification(sender, body)

        self.send_ok(f"Notification sent from {sender}")
    else:
        self.send_response(404)

        self.end_headers()
except Exception as e:
    print(f"Error handling request: {e}")

    self.send_response(500)

    self.end_headers()

    self.wfile.write(f"ERROR: {str(e)}\n".encode("utf-8"))
def vibrate_start(self, caller_name="Raspberry Pi"):
    subprocess.run(
        [
            "am",
            "broadcast",
            "-a",
            "nodomain.freeyourgadget.gadgetbridge.command.DEBUG_INCOMING_CALL",
            "-e",
            "caller",

```

```
        caller_name,
        "-p",
        "nodomain.freeyourgadget.gadgetbridge",
    ],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL,
)

def vibrate_stop(self):
    subprocess.run(
        [
            "am",
            "broadcast",
            "-a",
            "nodomain.freeyourgadget.gadgetbridge.command.DEBUG_END_CALL",
            "-p",
            "nodomain.freeyourgadget.gadgetbridge",
        ],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )

def send_notification(self, sender, body):
    subprocess.run(
        [
            "am",
            "broadcast",
            "-a",
            "nodomain.freeyourgadget.gadgetbridge.command.DEBUG_SEND_NOTIFICATION",
            "-e",
            "type",
            "GENERIC_SMS",
            "-e",
            "phoneNumber",
            "0123456789",
            "-e",
            "sender",
            sender,
        ],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )
```

```

        "-e",
        "subject",
        "Alert",
        "-e",
        "body",
        body,
        "-p",
        "nodomain.freeyourgadget.gadgetbridge",
    ],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL,
)

def send_ok(self, msg):
    self.send_response(200)
    self.send_header("Content-type", "text/plain; charset=utf-8")
    self.end_headers()
    self.wfile.write(f"OK: {msg}\n".encode("utf-8"))

# 一定時間アクセスがないかを監視するバックグラウンド関数
def idle_watcher():
    global last_access_time, server_instance
    while server_instance:
        time.sleep(10) # 10 秒ごとにチェック
        elapsed = time.time() - last_access_time
        if elapsed > IDLE_TIMEOUT_SEC:
            print(
                f"\n{IDLE_TIMEOUT_SEC}秒間アクセスがなかったため、サーバーを自動終了します。"
            )
            server_instance.shutdown()
            break

if __name__ == "__main__":
    server = HTTPServer(("0.0.0.0", 8080), RequestHandler)
    server_instance = server
    # アイドル監視スレッドをスタート
    watcher_thread = threading.Thread(target=idle_watcher, daemon=True)

```

```
watcher_thread.start()

print(
    f"Listening on 0.0.0.0:8080... (Auto-shutdown after {IDLE_TIMEOUT_SEC}s"
    " idle)"
)

try:
    server.serve_forever()
except KeyboardInterrupt:
    print("\nManual shutdown...")
finally:
    server.server_close()
```

サンプルコード（Raspberry pi 側）

```
import urllib.parse
import time
import requests

PHONE_URL = "http://{IP アドレス}:{ポート番号}"
# サンプルコード通りならポート番号は 8080

# 日本語やスペースを含める場合はエンコードが必要
# request.get の中に param を入れる形でも可能
sender_name = urllib.parse.quote("ラズパイ監視システム")
message_body = urllib.parse.quote("居眠りを検知しました。起きろ。")
caller_name = urllib.parse.quote("さぼり検知マン")

# 通知を送る
requests.get(f"{PHONE_URL}/notify?sender={sender_name}&body={message_body}")

time.sleep(1)
# 疑似電話（発信者名初期設定）
```

```
requests.get(f"{PHONE_URL}/start")

time.sleep(10)

requests.get(f"{PHONE_URL}/stop")

#疑似電話(発信者名変更)
requests.get(f"{PHONE_URL}/start?caller={caller_name}")
```